Scala

tipos fortes, código funcional
e concorrência na JVM

Seminario.scala

```scala
case class Seminario(
  tema: String,
  grupo: List[String]
)

val hoje = Seminario(
  "Scala",
  List("Pedro", "Luis",
       "Gabriel")
)
```

Pedro Henrique dos Santos · Luis Cesar · Gabriel Soares

Sistemas Distribuídos · 25.11.2026

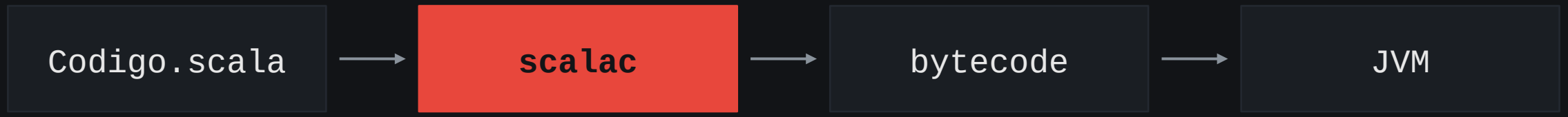
1

Tipos a seu favor

Pedro Henrique dos Santos

`o que é · opaque types · sealed`

Uma linguagem da JVM que mistura objetos e funções, com tipos verificados na compilação.



Roda em qualquer JVM e usa bibliotecas Java diretamente. Exemplos em Scala 3.

IDs que o compilador não deixa misturar

Ids.scala

```scala
object Ids:
  opaque type ClienteId = Long
  opaque type PedidoId  = Long
  def ClienteId(n: Long): ClienteId = n
  def PedidoId(n: Long): PedidoId = n
  extension (id: PedidoId)
    def proximo: PedidoId = id + 1

def cancelar(id: PedidoId) = ...

cancelar(PedidoId(7))     // ok
cancelar(ClienteId(42))   // não compila
```

```
$ scalac Ids.scala

Found:    ClienteId
Required: PedidoId
```

Custo zero
Em execução os dois são só Long. A diferença existe apenas para o compilador.

`extension`
Acrescenta métodos a um tipo sem alterá-lo nem criar um objeto embrulhando.

O compilador conhece todos os casos

Formas.scala

```scala
sealed trait Forma
case class Circulo(r: Double) extends Forma
case class Quadrado(l: Double) extends Forma
case class Triangulo(b: Double, h: Double)
  extends Forma

def area(f: Forma) = f match
  case Circulo(r)  => math.Pi * r * r
  case Quadrado(l) => l * l
```

```
$ scalac Formas.scala

Pattern Match
Exhaustivity Warning

match may not be
exhaustive.

It would fail on:
  Triangulo(_, _)
```

2

Código que vira linguagem

Luis Cesar

`erros como valores · given / using · by-name · futures`

A falha aparece no tipo de retorno

Divisao.scala

```scala
def div(a: Int, b: Int): Either[String, Int] =
  if b == 0 then Left("divisão por 0")
  else Right(a / b)

val r = for
  p <- div(100, 5)    // Right(20)
  q <- div(p, 0)      // Left: para aqui
yield q
// r == Left("divisão por 0")
```

Right
deu certo, segue

Left
falhou, o for para

Option
Some(valor) ou None, no lugar de null

O compilador escolhe o serializador

Codec.scala

```scala
trait Codec[T]:
  def codificar(t: T): String

case class Pedido(id: Long, total: Double)

given Codec[Pedido] with
  def codificar(p: Pedido) =
    s"""{"id":${p.id},"total":${p.total}}"""

def enviar[T](msg: T)(using c: Codec[T]) =
  "enviando " + c.codificar(msg)

enviar(Pedido(1, 99.9))
enviar(3.14)          // não compila
```

```
$ scalac Codec.scala

No given instance of
type Codec[Double]
was found for
parameter c
```

```
enviar(Pedido(1, 99.9))
enviando {"id":1,"total":99.9}
```

Esqueceu o serializador de um tipo? O erro aparece na compilação, e não quando a mensagem já está na rede.

Uma estrutura de controle em cinco linhas

Retry.scala

```scala
def comRetry[T](tentativas: Int)(bloco: => T): T =
  try bloco
  catch case e: Exception if tentativas > 1 =>
    println(s"falhou (${e.getMessage}), tentando de novo")
    comRetry(tentativas - 1)(bloco)
```

`=> T` o bloco é passado sem ser executado e roda de novo a cada tentativa.

Cliente.scala

```scala
comRetry(3) {
  chamarServidor()
}
```

```
$ scala Cliente

falhou (timeout), tentando de novo
falhou (timeout), tentando de novo
resposta do servidor
```

Futures rodam em paralelo

Paralelo.scala

```scala
val f1 = Future { Thread.sleep(200); 21 }
val f2 = Future { Thread.sleep(200); 2 }

val total = for a <- f1; b <- f2 yield a * b

Await.result(total, 2.seconds)   // 42
```



0,22 s

tempo medido, e não 0,4 s: as duas tarefas rodaram juntas em threads do pool

3

Na prática

Gabriel Soares

atores · spark · kafka · críticas

Mensagens no lugar de locks



Cada ator processa uma mensagem por vez e ninguém toca no estado do outro. Sem estado compartilhado, não há condição de corrida.

Akka

Toolkit de atores do ecossistema Scala. O ator do outro lado pode estar em outra máquina.

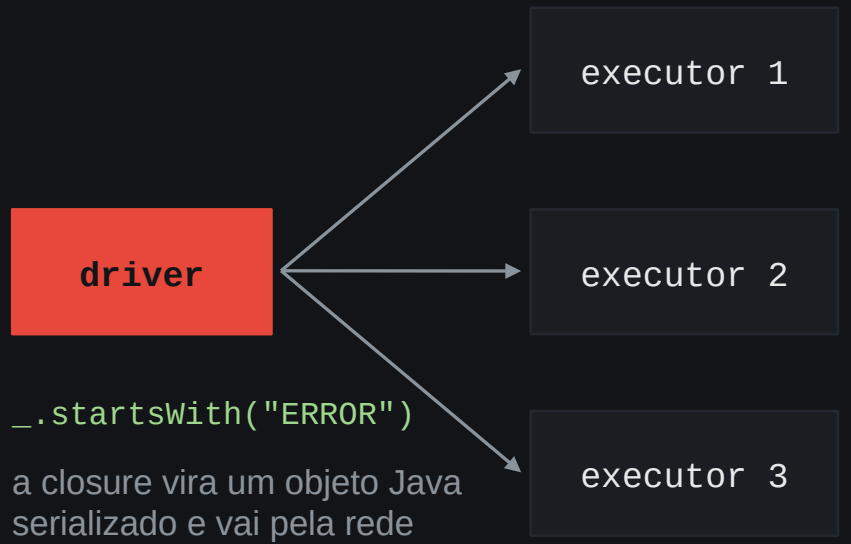
Desde 2022 tem licença comercial. O Apache Pekko é o fork aberto.

O código viaja até os dados

LogMining.scala

```scala
val lines  = spark.textFile("hdfs://...")
val errors = lines.filter(_.startsWith("ERROR"))
errors.persist()

errors.filter(_.contains("HDFS"))
      .map(_.split('\t')(3))
      .collect()
```



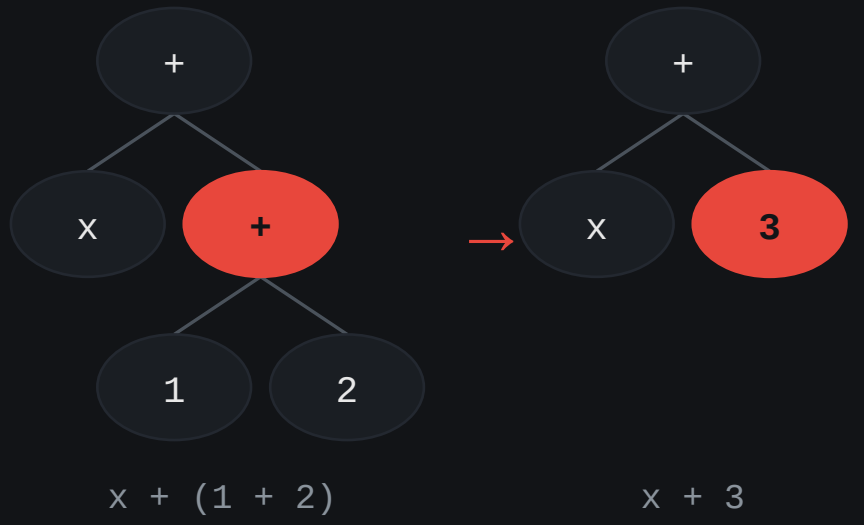
“Escolhemos Scala pela combinação de concisão (conveniente para uso interativo) e eficiência (por causa da tipagem estática).”

Zaharia et al., artigo original do Spark (RDDs), NSDI 2012

O otimizador do Spark SQL é pattern matching

Catalyst.scala

```scala
tree.transform {
  case Add(Literal(a), Literal(b)) => Literal(a + b)
  case Add(left, Literal(0))       => left
  case Add(Literal(0), right)      => right
}
```



A consulta SQL vira uma árvore de objetos imutáveis (case classes). Cada regra de otimização é um case que procura um formato na árvore e devolve uma árvore nova. É o pattern matching da parte 1 no coração de um sistema distribuído.

Regras tiradas do artigo do Spark SQL (Armbrust et al., SIGMOD 2015). Uma versão mínima roda no exemplos.scala.

Kafka nasceu em Scala e migra para Java

2011

LinkedIn abre o código. Broker e clientes em Scala.

0.8.2 / 0.9

Producer e consumer reescritos em Java.

2.0

Clientes Scala removidos.

4.0

Só Scala 2.13. Código novo, como o KRaft, em Java.

Por que os clientes saíram do Scala

- versões do Scala incompatíveis entre si
- classes Scala vazando na API usada em Java
- Javadoc ruim e stack traces difíceis de ler

~8% do repositório ainda é Scala

Mas o núcleo do broker continua lá:

`ReplicaManager.scala`
`KafkaApis.scala` `LogManager.scala`

Segurança tem custo

+	−
Erros de tipo e casos esquecidos param na compilação	Curva de aprendizado alta
Imutabilidade facilita concorrência	Compilação lenta e comunidade pequena
Todo o ecossistema Java disponível	Versões incompatíveis entre si, o motivo de o Kafka ir para Java

Perguntas?

Scala compensa quando correção e concorrência importam mais que a velocidade de escrever.

Pedro · Luis · Gabriel

Para estudar cada conceito

opaque types	docs.scala-lang.org/scala3/book/types-opaque-types.html
extension methods	docs.scala-lang.org/scala3/book/ca-extension-methods.html
sealed e pattern matching	docs.scala-lang.org/scala3/book/types-adts-gadts.html
Option e Either	docs.scala-lang.org/scala3/book/fp-functional-error-handling.html
given / using	docs.scala-lang.org/scala3/book/ca-context-parameters.html
by-name	docs.scala-lang.org/tour/by-name-parameters.html
Futures	docs.scala-lang.org/overviews/core/futures.html
atores (Pekko)	pekko.apache.org/docs/pekko/current/typed/actors.html
Spark (RDDs)	Zaharia et al., NSDI 2012 · usenix.org
Catalyst	Armbrust et al., SIGMOD 2015 · blog da Databricks
Kafka em Java	cwiki.apache.org/confluence/display/KAFKA/Client+Rewrite